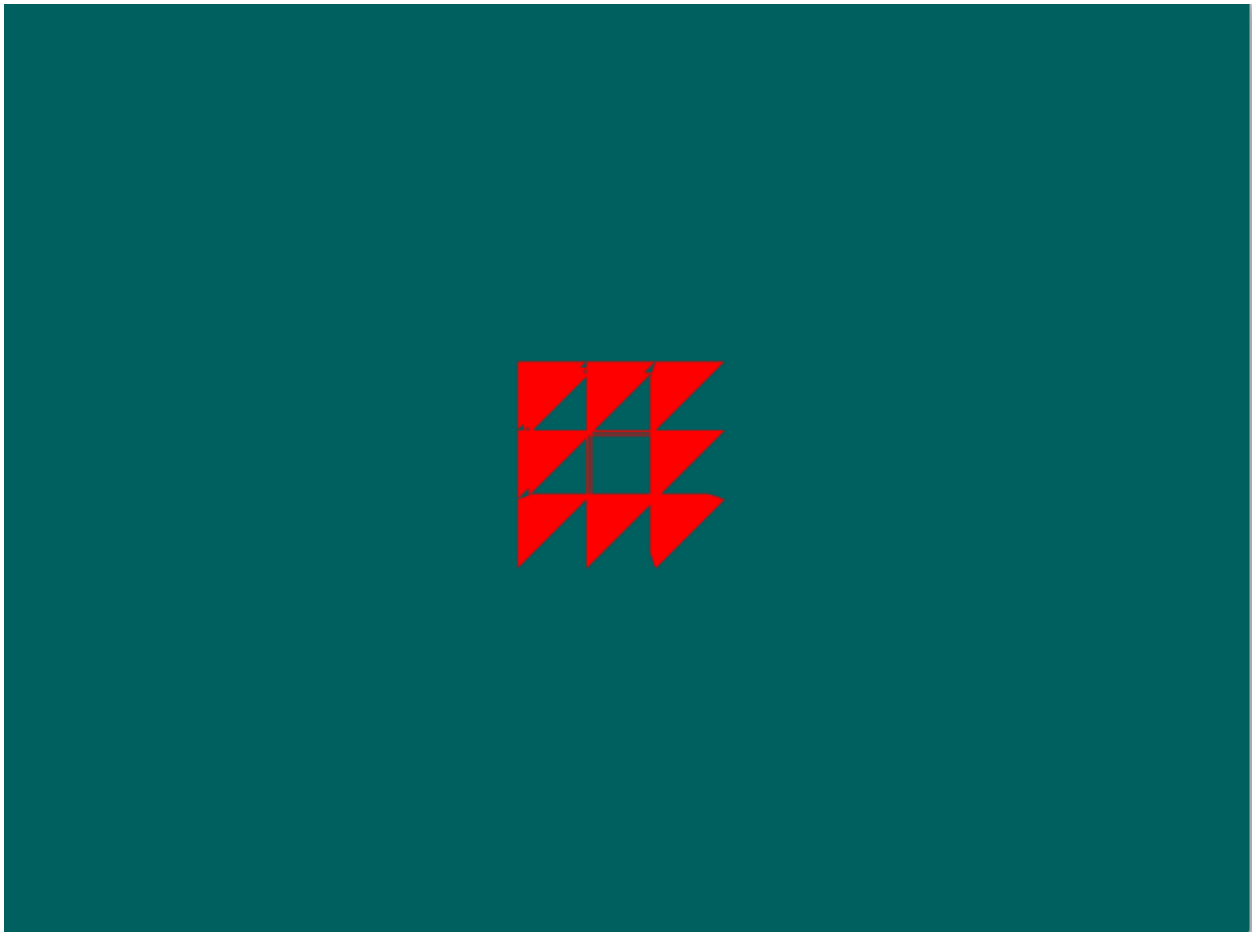
******* Danielle (Mckenna) McIntyre Section *******
Files I worked on: MengerSponge.ts

I worked on implementing the functions inside of MengerSponge.ts, mainly focusing on the functions addCube and generateSponge. Initially I did a bit of research on mengerSponge code by using sources like Wikipedia and StackExchange to get a general idea on what my code needed to do, and what types of formulas I could use as a reference for building my functions.

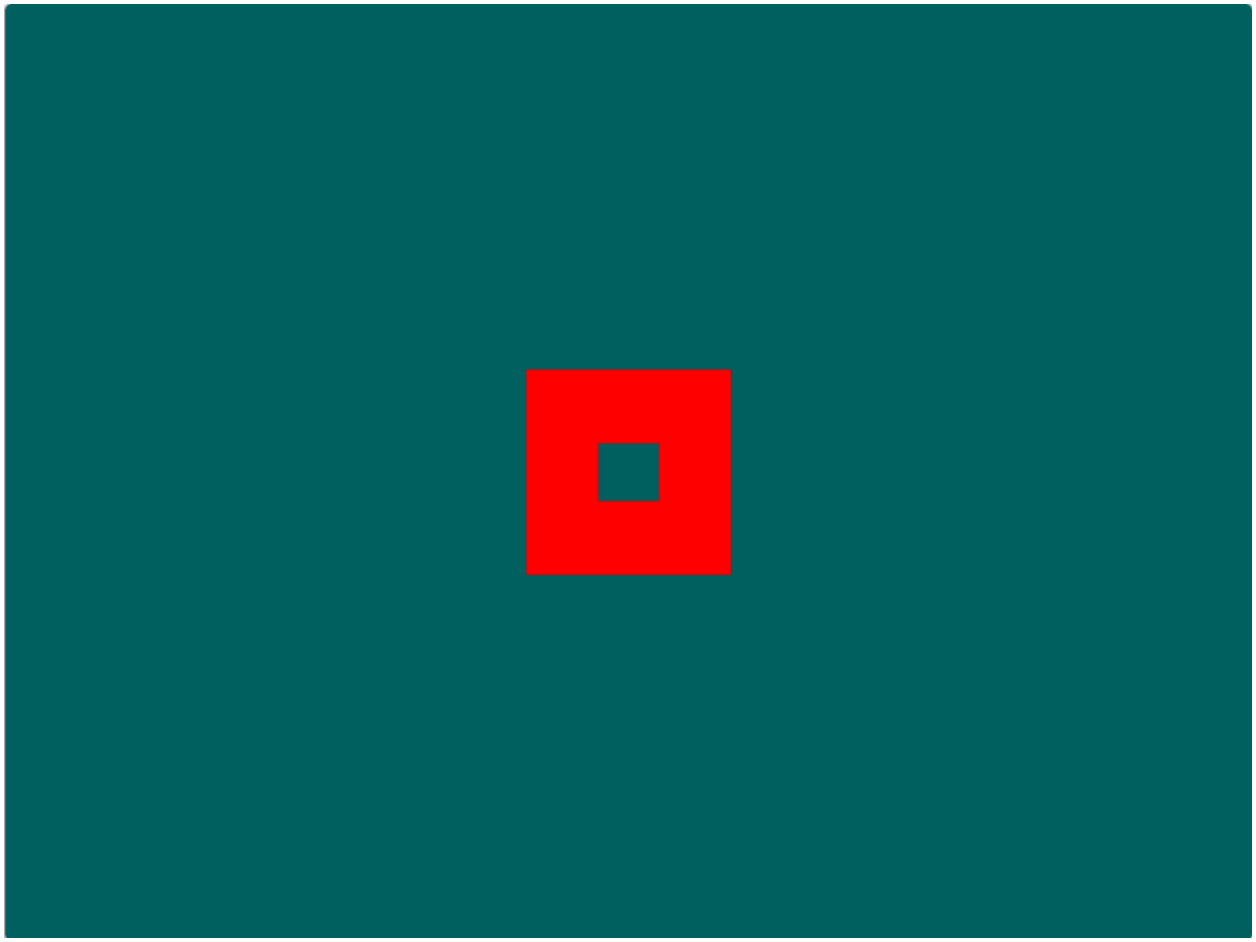
To build the cube, I used the variables i, j, and k to represent the X, Y, and Z axis. They were iterated from -1 to 1 in their respective loops, which were all nested together, to generate the 27 possible positions of the 3x3x3 cube, centered at the point (0,0,0). Eventually I found that working with the formula (Math.abs(i) + Math.abs(j) + Math.abs(k)) <= 1 would work the best in finding the inner subcubes, the dead center of the cube and the centers of each face, and remove them.

This loop works with generateSponge() by dividing the cube into the 27 possible subcubes when the level is set higher than 0, using the math formula mentioned previously to remove the 7 empty spaces. Every time the recursion operates, the size is divided by 3, which would eventually create the infinitely repeating sponge division as the levels increased.

My main issue revolved around defining the cube itself, as in my initial test run of the code, the geometry displayed incorrectly, as shown below:



Instead of solid squares that represent one side of the cube face for level 1, it instead came out as several half triangles overlapping each other and leaving large gaps where the other half triangles should have been set. I used AI to make sure that my implementation of the Math.abs variables were correct, thinking that might have been the cause of this error, as well as double check the rest of my code for any issues. I then realized the error was in the addCube() function, as my vertices representing the back, left, and right faces had been defined out of order, rather than the counter-clockwise positioning that WebGL uses by default. In order to fix this, I reworked the cube geometry into their own distinct vertices to ensure each triangle would be positioned in the correct angle, and reworked the faceVertices array so that the triangle faces were specifically listed in counter-clockwise order. The final base result can be seen here:



****** Emily Tutt Section ******
Files I worked on: Shader.ts, Gui.ts

I worked on implementing face shading for the cube using the colors described in the project description and the initial shading information based on lighting. I read through the course slides related to OpenGL Shaders and OpenGL documentation to understand the pipeline for shading.

The fragrant shader block works by normalizing the vectors containing the face normals of the cube (N) and the lighting (L) to get a unit vector. These are necessary for determining which face of the cube each pixel is in. Next, the lighting of each side is calculated using a Lambertian diffuse lighting model. This variable, diffuseL, returns a float which is higher if the point is facing more towards light and lower if facing more away from the light source.

Using the face coloring table from the project description, the base color for each component is evaluated and then multiplied by the diffuseL to determine what shade of that color it needs to be based on the lighting and position.

```
void main () {
```

```
vec3 N = normalize(normal.xyz);
vec3 L = normalize(lightDir.xyz);

//keep lighting for shading
float diffuseL = max(dot(N,L), 0.0);

//determine the color based on the face of the cube
vec3 baseColor = vec3(0.0, 0.0, 0.0);

if(abs(N.x) > 0.9){
    baseColor = vec3(1.0, 0.0, 0.0);
}
else if(abs(N.y) > 0.9){
    baseColor = vec3(0.0,1.0,0.0);
}
else{
    baseColor = vec3(0.0, 0.0, 1.0);
}

//compute the shade of the base color based on lighting conditions
gl_FragColor = vec4(baseColor * diffuseL, 1.0);

}
```

I also worked on implementing the extra credit CTRL + S shortcut to save the menger sponge geometry to an OBJ file. I referred to the wikipedia page for a .obj file to understand the format and stack overflow for help on writing data to a .obj file. These functions allow the viewer to locally download a .obj file containing the currently displayed sponge's geometry.

This file stores the vertices, normals, and faces (in this case the triangle polygons). The following code activates the .obj writing when the ctrl key and s key are clicked together:

```
if(key.ctrlKey && key.key == "s") {
  this.saveOBJ(this.sponge);
  return;
}
```

This calls the new function saveOBJ() which takes the current sponge as a parameter. This function then dissects the vertices (sponge.positionsFlat), normals (sponge.normalsFlat), and triangle faces (sponge.indicesFlat). It then creates an array to store the lines of data and loops through each respective set of points and writes them in the format required for .obj ('v' for vertices, 'vn' for normals, and 'f' for indices or triangle face vertices).

Then a blob object is created. A Blob is basically a temporary raw data file that allows the file to be downloaded. It uses the url as an identifier and creates an "<a>" element which points to the url. The file is then downloaded and revokes the blob url so it doesn't sit in memory. I was very unfamiliar with this part, so I referred to Stack Overflow and ChatGPT to explain the pipeline.

*****Rudwika Manne*****

World Transformer – Menger Sponge

In this project, my role was the *World Transformer*. My responsibility was to implement the model transformation matrix for the Menger sponge so that the object can be properly positioned and animated in the 3D scene. The transformation is handled in the `uMatrix()` function inside the `MengerSponge.ts` file.

The `uMatrix()` function generates a model matrix that defines how the sponge is transformed before it is rendered. First, the matrix is initialized as an identity matrix. Then, a small scaling transformation is applied so the sponge appears slightly smaller and fits well within the camera view. After scaling, a rotation transformation is applied around the Y-axis. The rotation uses the current time to continuously rotate the sponge, which makes the animation dynamic when the program runs.

This transformation matrix is sent to the graphics pipeline and combined with the view and projection matrices in the vertex shader. The final transformation converts the sponge's vertices from model space to clip space, allowing WebGL to correctly display the object in the scene.

By implementing the `uMatrix()` function, the Menger sponge now rotates smoothly in the world coordinate system while maintaining the correct position and scale. This demonstrates how model transformations control the placement and motion of objects in a 3D graphics application.

*****Daniel Kee Tam *****

Files I worked on include: Gui.ts, MengerSponge.ts

My main task was to implement FPS camera navigation and keyboard controls in the Gui.ts file. During implementation, I used the WebGL utilities documentation, the project description, and Gemini to supplement my learning. During this section, my learning focus was understanding how to manipulate the camera's orthonormal frame, basically the eye, look, and up vectors.

To implement the mouse rotation portion, I first tried a basic two-axis rotation, but then realized the project's description would best be fit through the cross-product method. The logic within my code works by applying the mouse's screen distance to the camera's right and up vectors and using that to create a world-space direction vector. Afterwards, I calculated a perpendicular rotation axis using a cross product of the camera's forward direction and the mouse vector. This allowed the camera to swivel by the rotation_speed variable.

For the keyboard movement logic (WASD, arrow keys, and numbers for different Menger sponge levels), one important detail I noted during implementation was using the "true" flag for

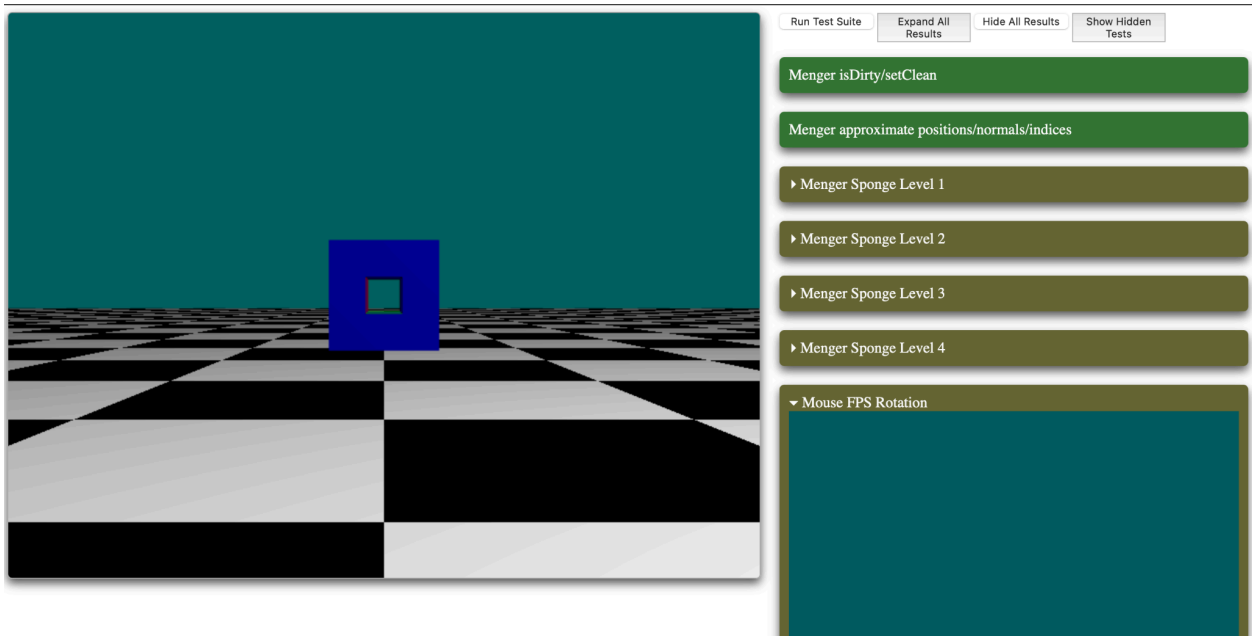
the offsetTarget parameter. This was especially important since the project wanted both the target and the camera's "eye" to move together to strafe like in an FPS.

One issue I faced was a visual glitch where the right and left sides of the fractal cube appeared transparent. This was only possible to initially see with camera controls. So while implementing, I was the first one to be able to see the right side of the cube clearly. After consulting group members, I realized quickly that the issue was an issue with the declaration of the vertices for the faces. They treated those faces as if they were inside faces, making them transparent. I reordered the vertices in the faceVertices array, and it led to normals pointing outward, rendering the faces correctly.

Lauren Camper
Files I worked on: App.ts, Shaders.ts

My main focus for this project was the floor implementation. To give the scene a visual reference point, I implemented a flat ground plane underneath the Menger sponge. The floor was created using four vertices that form a large square at y = -2.0, and two triangles were used to render the surface. I created separate buffers and a vertex array object for the floor so that it could be rendered independently from the sponge geometry. The floor data is sent to the GPU using WebGL buffers and is drawn after the sponge in the render loop.

The floor uses its own shader program. The vertex shader transforms the floor vertices into screen space using the world, view, and projection matrices and passes lighting and world position information to the fragment shader. The fragment shader generates a checkerboard pattern by dividing the world x and z coordinates into grid cells of size 5 × 5 units and alternating between black and white colors. Basic diffuse lighting is also applied so the floor is shaded based on the direction of the light source.



A screenshot of the final program shows the Menger sponge rendered above a checkerboard floor plane. The floor extends far in the x and z directions and provides a stable reference surface when moving the camera around the scene.

There were no major bugs or issues with the floor rendering. However, because the plane is implemented as a very large square instead of a mathematically infinite plane, it may eventually end if the camera moves extremely far away.